Personal Ledger

A personal finance tracker — income, expenses, budgets, loans, and analytics — backed by Supabase Auth + Postgres, deployable for free.

Stack: Vite + React (plain JSX), Recharts, Supabase. No backend of your own.

---

1. Create a Supabase project

1. Go to supabase.com and create a free account and project.
2. Open **SQL Editor** → **New query** and run the files in `migrations/` **in numerical order** (`001` through `004`). They're idempotent — re-running them is safe and changes nothing.
3. Go to **Project Settings** → **API**. Copy the **Project URL** and the **anon public** key.
4. Under **Authentication** → **URL Configuration**, add your redirect URLs. Password reset and email confirmation links bounce without this:
   - `http://localhost:5173/**` for local development
   - `https://your-app.vercel.app/**` once deployed

2. Configure

```
cp .env.example .env      # then paste in your Project URL + anon key
npm install
npm run dev
```

Sign up with your email and start logging. If the app shows **"Not configured"**, the env vars aren't reaching it — check `.env` exists and restart the dev server (Vite only reads env vars at startup).

3. Deploy

Vercel or **Netlify**, same shape:

1. Push to a GitHub repo.
2. Import the repo on vercel.com.
3. Add `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` as environment variables.
4. Deploy.

> **Set the env vars before the first build.** They're inlined at build time, not read at runtime. A build without them produces an app that only renders the "Not configured" screen.

Pushing updates without touching your data

Deploys cannot wipe your database. The frontend is a static build — Vercel runs `npm run build` and serves `dist/`. Nothing in that pipeline can reach Postgres. Your data lives entirely in Supabase and is untouched by a `git push`.

The only way to lose data is to run a destructive statement in the SQL editor yourself. The rules that prevent that are in `migrations/README.md`; the short version:

- Never edit a migration that has already been run — add a new numbered file.
- Every statement idempotent (`if not exists`, `drop policy if exists`).
- No `drop table`, no `truncate`, no unqualified `delete`.
- Adding a column? Give it a default or allow nulls.

Schema changes are a separate, deliberate act from deploying code. Run the new migration in the SQL editor, then push the code that uses it.

Letting friends test it

Sign-up is open — anyone with the URL can create an account. Row-level security scopes every table by `auth.uid()`, so each tester only ever sees their own rows; there's no shared state between accounts. New accounts seed a starter set of categories on first load.

Testers can hit **Feedback** in the sidebar to send you a note. It writes to the `feedback` table, which is deliberately **insert-only**: they can send, but nobody can read it back through the app — not even their own. Read it in **Table Editor → feedback** in the Supabase dashboard.

Two things worth knowing before you invite people:

- **Email is rate-limited.** Supabase's built-in sender allows only a few emails per hour, shared across confirmations and password resets. Fine for a handful of testers trickling in; it will bite if several sign up at once. To lift it, add custom SMTP (Resend, Brevo) under **Authentication → SMTP Settings** — a dashboard change, no code change.
- **Consider turning off email confirmation** while testing (**Authentication → Providers → Email**) so testers aren't blocked on a rate-limited inbox.

To close sign-ups later, disable the email provider in the same panel and invite people individually with **Authentication → Users → Invite**.

Keeping Supabase from pausing

The free tier pauses a project after 7 days with no API traffic. Data isn't deleted, but the app goes offline until you resume it from the dashboard. Point a free UptimeRobot HTTP monitor at your Supabase project URL every few days, or run a scheduled GitHub Action that `curl`s it.

How it's put together

```
migrations/          numbered, idempotent SQL — run in order
src/
  theme.js           design tokens + global CSS, applied to :root
  lib/
    format.js        money + date helpers
    analytics.js     all derived data — pure functions, no React
    chartTheme.js    validated chart colours
```

```
    supabaseClient.js
  hooks/
    useLedgerData.js every Supabase read and write
  components/        Shell (nav, feedback) + primitives
  views/             Overview, Analytics, Register, Budgets, Loans,
Settings
  auth/              sign in/up, reset request, reset confirm
```

Responsive layout has one switch, not many: `BREAKPOINT` (900px) in `theme.js`, read by CSS media queries there and by the `useIsNarrow` hook in `hooks/useIsNarrow.js`. Below it the sidebar becomes a bottom tab bar, every grid collapses to one column, and the two wide tables (register, month comparison) re-render as stacked cards rather than scrolling sideways.

Three things are easy to undo by accident:

- Grids use `minmax(min(300px, 100%), 1fr)`, via the `AutoGrid` primitive. Plain `minmax(300px, 1fr)` keeps its floor when the container is narrower and pushes the page sideways.
- Controls must compute to **16px** on phones or iOS zooms the page on focus. Sizes are set inline, so the override in `theme.js` needs `!important`.
- Charts are wrapped in `<ScrollX fluid>` so they reflow. Their box is *taller* on a phone, not shorter — the legend wraps to a second line and `ScrollX` clips whatever overflows.

`lib/analytics.js` holds the month-over-month comparison, trends, heatmap, budget burn-down, forecasting and recurring-charge detection. It's deliberately free of React and Supabase imports, so the maths can be checked in isolation.

Forecasting projects month-end spend from the burn rate, but extrapolates only *variable* spend — detected recurring charges are added at their typical amount instead of being scaled up. Without that, a rent payment on the 3rd makes the whole month look catastrophic.

Recurring detection is a heuristic over data already loaded: 3+ charges with similar descriptions, roughly monthly gaps (25–35 days), and amounts stable within 15%. Nothing is stored — it's recomputed each load.

Chart colours in `lib/chartTheme.js` were validated for colour-blind separation and contrast against the dark surface. Don't hand-tweak them; if they need to change, re-run a palette validator. The accent (`volt`) is deliberately *not* a series colour — it's a UI/text token.

Notes

- Amounts are Ghanaian cedis (₵) in the UI only; the database stores plain numbers. Change `fmtMoney` in `src/lib/format.js` for a different currency.
- `transactions.amount` is signed: positive = income, negative = expense.
- Categories are denormalised text on `transactions` and `budgets`, with no foreign key to `categories`. Renaming a category won't move existing rows.